Engineered Degradation Features for Remaining Useful Life Prediction on C-MAPSS: Phase 1 Machine-Learning Baselines

Gokul VKS, Milind Bansal, Isha Tomar, Yashi Agrawal and Mohammad Affan Anas
Newton School of Technology, Rishihood University
Code and results: https://github.com/vks-g/cmapss-rul-hybrid

***Abstract*—Remaining useful life (RUL) prediction estimates how many operating cycles an engine has left before it fails. This report presents Phase 1 of a project that compares engineered degradation features with sequence models on the NASA C-MAPSS turbofan data: feature-based machine-learning baselines on subset FD001, under a protocol that Phase 2 will reuse. Training engines are split by engine, every preprocessing step is refitted inside each training fold, the RUL target is uncapped, and each of the 100 official test engines is scored once at its last cycle. From regime-normalized sensor readings we build 155 causal predictors (rolling means, slopes and changes since each engine's first cycle) and tune Random Forest, XGBoost and Elastic Net regressors on them and on 25 raw inputs. The engineered Random Forest reaches a test RMSE of 27.55 cycles and an MAE of 17.67 cycles, and all six models lie within five RMSE cycles of each other. The engineered features roughly halve the tree models' error near failure (RMSE 7.52 and 5.62 against 13.50 and 14.32 with raw inputs) but overestimate young engines, and a single engine contributes 92% of the Random Forest's NASA score. Capping the training target at 125 cycles lowers the Random Forest's RMSE to 16.21 with engineered and 18.66 with raw inputs, reversing their order.**

***Index Terms*—remaining useful life, predictive maintenance, C-MAPSS, feature engineering, random forest, XGBoost, elastic net**

I. Introduction

An aircraft engine that fails in service grounds the aircraft and can endanger it, while an engine that is overhauled too early wastes useful life. Predictive maintenance resolves this trade-off by estimating, from the engine's own sensor history, its remaining useful life (RUL): the number of operating cycles left before failure [1], [2]. Because run-to-failure records of real engines are rare, most data-driven work uses NASA's C-MAPSS simulation data [3], [4], in which every training engine is observed until it fails and every test engine stops at an unknown point before failure.

This project asks whether hand-engineered degradation features are enough for RUL prediction, or whether models that learn directly from raw sensor sequences do better. Fixed rolling summaries of recent history may miss long degradation trajectories and interactions whose timing differs from engine to engine, which sequence models are designed to capture [5]. Answering the question fairly needs a strong feature-based baseline and an evaluation protocol that both model families share. Published C-MAPSS results are hard to compare because they differ in subsets, targets and protocols [6].

Phase 1, reported here, builds that baseline on FD001, the subset with one operating condition and one fault mode. Its contributions are:

- a leakage-controlled protocol, fixed for Phase 2: engine-grouped cross-validation, preprocessing refitted inside every training fold, an uncapped RUL target, and scoring of each official test engine once at its last cycle with RMSE, MAE and the asymmetric NASA score, overall and by degradation stage;
- a causal feature set of 155 predictors per cycle (regime-normalized sensor readings with rolling means, slopes and changes from each engine's first cycle), compared with 25 raw inputs;
- Random Forest, XGBoost and Elastic Net models tuned under this protocol and scored on the 100 official FD001 test engines;
- an error analysis showing that the engineered features roughly halve the tree models' error near failure but overestimate young engines, that the NASA score is dominated by single engines, and that capping the target reverses the raw-versus-engineered comparison.

Section II reviews related work and Section III the data. Section IV defines the problem and the protocol, Sections V and VI describe the features and models, and Section VII reports the results. Section VIII concludes and outlines Phase 2.

II. Literature Review

A. Prognostics and Remaining Useful Life

Condition-based maintenance schedules maintenance from the measured condition of a machine rather than from a fixed calendar, and prognostics, estimating how long a component can keep operating, is its forward-looking half [1]. The quantity to estimate is the remaining useful life (RUL): the number of operating cycles left before failure. Reviews group RUL methods into physics-based models, which need an explicit model of the failure mechanism; data-driven models, which learn degradation from condition-monitoring data; and hybrids of the two [2]. Statistical data-driven approaches, from regression on degradation signals to stochastic-process models, are surveyed by Si et al. [7]. Lei et al. [2] divide the data-driven pipeline into data acquisition, health-indicator construction, health-stage division and RUL prediction. The last two steps

motivate our reporting of errors separately for the early, middle and late stages of degradation.

B. The C-MAPSS Benchmark

Run-to-failure data from real engines is scarce, so simulation fills the gap. Saxena et al. [3] generated turbofan degradation data with the C-MAPSS simulator. Each engine starts from a random, unknown level of initial wear; a fault then reduces the flow and efficiency of engine modules at an exponential rate, and the engine fails when a health index built from its operating margins reaches zero. The resulting data set [4] contains four subsets, FD001 to FD004, that combine one or six operating conditions with one or two fault modes. For test engines it gives only a truncated trajectory and the true RUL at its last cycle. Saxena et al. [3] also defined the asymmetric score used in the PHM08 challenge, which penalizes late predictions more heavily than early ones because an unexpected failure costs more than early maintenance; a broader set of prognostic metrics is discussed in [8]. Six years later, Ramasso and Saxena [6] reviewed more than seventy publications on these data. They found that the absence of benchmark results, together with misunderstandings about how the subsets relate, made reported results hard to compare, and they named sensor noise, changing operating conditions and multiple fault modes as the main difficulties of the benchmark.

C. Feature-Based Machine Learning

Early solutions to the PHM08 challenge already show the two directions the field has followed. Wang et al. [9] built a library of degradation patterns from training engines and estimated a test engine's RUL from the lives of the most similar patterns, while Heimes [10] trained a recurrent neural network and placed second in the competition. Later work mapped sensor values to RUL directly. Khelif et al. [11] used support vector regression with wrapper-based sensor selection, removing the need to estimate degradation states or a failure threshold, and Mathew et al. [12] compared a range of classical regressors on the NASA turbofan data. Random forests have also served both as accurate regressors and as feature selectors for a neural network [13], [14]; Section VI discusses these studies alongside our models.

Our Phase 1 baselines follow this direct-regression line with three model families chosen for complementary behavior. Random forests [15] average many decorrelated regression trees, capturing non-linear sensor interactions with little tuning. XGBoost [16] adds trees one at a time, each fitted to the gradient of the loss of the current ensemble [17], with explicit regularization of tree complexity; it usually trades more tuning for higher accuracy. The elastic net [18] is a linear model with combined ℓ_1 and ℓ_2 penalties whose grouping effect keeps strongly correlated predictors together, which suits the redundant C-MAPSS sensors and gives an interpretable linear reference. All three consume fixed-length summaries of each engine's recent history, such as rolling means and slopes, rather than the raw sequence.

TABLE I
C-MAPSS SUBSETS. ENGINE AND ROW COUNTS ARE TAKEN FROM THE RAW FILES; LIFETIMES ARE THE FINAL CYCLES OF THE TRAINING ENGINES.

Subset	Cond.	Faults	Engines		Lifetime	
			Train	Test	Median	Range
FD001	1	1	100	100	199	128–362
FD002	6	1	260	259	199	128–378
FD003	1	2	100	100	220.5	145–525
FD004	6	2	249	248	234	128–543

D. Sequence Models and the Research Gap

Deep learning removes the hand-crafted summary step. Convolutional networks applied to fixed-length windows of raw sensor readings [19], [20] and long short-term memory (LSTM) networks [5], [21] have reported lower errors on C-MAPSS than the regression baselines they were compared with. Zheng et al. [5] argue that features built from sliding windows do not fully use the order of the sensor sequence. Many of these models are also trained against a piecewise-linear target that is held constant early in life, when degradation is not yet visible in the sensors, and decreases linearly afterwards [5], [10], [19], [20].

Two gaps motivate this project. First, fixed rolling features may miss long degradation trajectories and interactions whose timing differs from engine to engine, which is what sequence models are designed to capture. Second, as Ramasso and Saxena [6] showed, published comparisons often differ in the subsets, targets and evaluation protocols they use. Phase 1 therefore builds strong feature-based baselines under a protocol that stays fixed for Phase 2: splits by engine rather than by row, an uncapped RUL target with the capped target treated as an ablation, and RMSE, MAE and the asymmetric score reported both overall and by degradation stage.

III. Dataset and Exploratory Analysis

A. The C-MAPSS Data Set

We use the NASA C-MAPSS turbofan degradation data [4], generated with the simulator and damage-propagation model of Saxena et al. [3]. It has four subsets that differ in the number of operating conditions and fault modes (Table I). Every subset has a training file, in which each engine is observed from an unknown initial wear level until failure, and a test file, in which each trajectory stops at some point before failure. A separate file gives the true RUL at the last recorded cycle of each test engine. Engine identifiers are local to each subset and split. FD001 is the primary subset of this project; the other three show how the difficulty grows when operating conditions and fault modes are mixed, which Ramasso and Saxena [6] identify as the main obstacles to comparing results on this benchmark.

The fault mode is degradation of the high-pressure compressor (HPC) in FD001 and FD002; FD003 and FD004 add fan degradation. FD001 contains 20,631 training rows and 13,096 test rows. Each row is one engine cycle with 26 columns: engine identifier `unit`, cycle index `cycle`, three operating settings `op_1`–`op_3` (altitude, Mach number and throttle resolver angle) and 21 sensor channels $s_1, \ldots, s_{21}$.

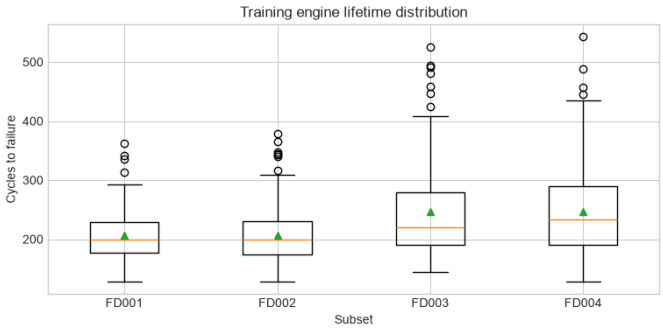


Fig. 1. Distribution of training-engine lifetimes per subset. Triangles mark the mean.

The sensors record temperatures, pressures, shaft speeds, ratios and bleed flows at different engine stations; for example s_2, s_3 and s_4 are commonly identified with the LPC, HPC and LPT outlet temperatures and s_{11} with the HPC outlet static pressure [3]. RUL is not one of the columns. The training target is the engine's final cycle minus the current cycle, and test engines are scored only at their last observed cycle against the supplied RUL.

B. Exploratory Findings

The analysis below uses training trajectories only, so that no test information shapes later modeling choices. It describes the data; the transforms it motivates are fitted inside each training fold, as described in Section V.

1) Lifetimes differ widely between engines: Fig. 1 shows that FD001 lifetimes range from 128 to 362 cycles around a median of 199, and the two-fault subsets are longer and more spread out, reaching 525 and 543 cycles. Because an engine's rows are strongly related to each other, a row-level split would place cycles of the same engine on both sides of the split. All evaluation is therefore split by engine. The long right tails also mean that the uncapped RUL target spans a wide range, which inflates absolute errors on early-life cycles.

2) Some sensors carry no information in FD001: Six FD001 sensors are exactly constant across all training rows: s_1, s_5, s_{10}, s_{16}, s_{18} and s_{19}. They cannot separate healthy from degraded engines, so the default selection rule removes them. Three further channels, s_6, s_8 and s_{13}, vary very little relative to their mean, but small variation is not the same as no information, and they are retained. FD003 has five constant sensors, and FD002 and FD004 have none, because a sensor that is fixed within one operating condition still changes between conditions.

3) Operating conditions dominate raw sensor levels: The operating settings of FD002 and FD004 take only six combinations, the six simulated flight conditions (Table II); every training row lies within 0.006 of its condition's mean settings. Fig. 2 shows the consequence. In FD001 each sensor has a single, roughly bell-shaped distribution, whereas in FD002 the same sensors split into separate modes, one or more per condition, spread over a range much wider than the change caused by degradation. Pooled statistics such as correlations are therefore confounded by the operating condition in these subsets. This finding motivates normalizing each sensor within its learned operating regime before computing degradation features.

TABLE II
THE SIX OPERATING CONDITIONS OF FD002 AND FD004: MEAN SETTINGS OF THE TRAINING ROWS IN EACH CONDITION AND ITS SHARE OF THE TRAINING ROWS.

op_1	op_2	op_3	FD002	FD004
0	0.00	100	15.0%	15.1%
10	0.25	100	15.1%	15.1%
20	0.70	100	15.1%	14.8%
25	0.62	60	14.9%	14.9%
35	0.84	100	15.0%	15.0%
42	0.84	100	25.0%	25.1%

4) Degradation is slow early and accelerates near failure: To compare engines of different lengths, Fig. 3 averages FD001 sensor readings in bins of the fraction of each engine's observed life. The informative sensors s_2, s_3, s_4 and s_{11} change little during roughly the first half of life and then rise increasingly steeply until failure. This shape is consistent with the exponential fault growth of the simulation model [3]. It has two implications. First, the level and the recent slope of a sensor both carry information, which motivates the rolling mean and slope features. Second, early-life cycles look alike in the sensors while their true RUL differs widely, so errors on early cycles are expected to be large under the uncapped target; this is why errors are broken down by degradation stage and a capped target is checked as an ablation (Section VII). The life fraction uses each training engine's final cycle and is used here only for description; it is never a model input. Individual engines are noisier than these averages, and they start from different levels because initial wear differs.

IV. Methodology

This section defines the prediction task and the evaluation protocol shared by every Phase 1 model; Phase 2 keeps it unchanged. Fig. 4 summarizes the protocol. The features and models it evaluates are described in Sections V and VI.

A. Problem Formulation

Engine u is observed at cycles $t = 1, \ldots, T_u$ with predictor vector $\mathbf{x}_{u,t}$. A training engine runs until it fails after cycle T_u, so its RUL at cycle t is $y_{u,t} = T_u - t$. A test engine is observed only up to a cycle τ_u before failure, and its true RUL is given for that cycle alone. A predictor may use only the engine's own history, $\hat{y}_{u,t} = f(\mathbf{x}_{u,1}, \ldots, \mathbf{x}_{u,t})$, which matches how RUL would be estimated during operation. The project brief asks for the target to be capped only when stated, so the uncapped $y_{u,t}$ is the primary target. A target capped at 125 cycles, $\min(y_{u,t}, 125)$, which much of the literature uses [5], [10], is treated only as an ablation.

B. Engine-Grouped Validation

Consecutive cycles of one engine are nearly identical, so splitting rows at random would put cycles of the same engine

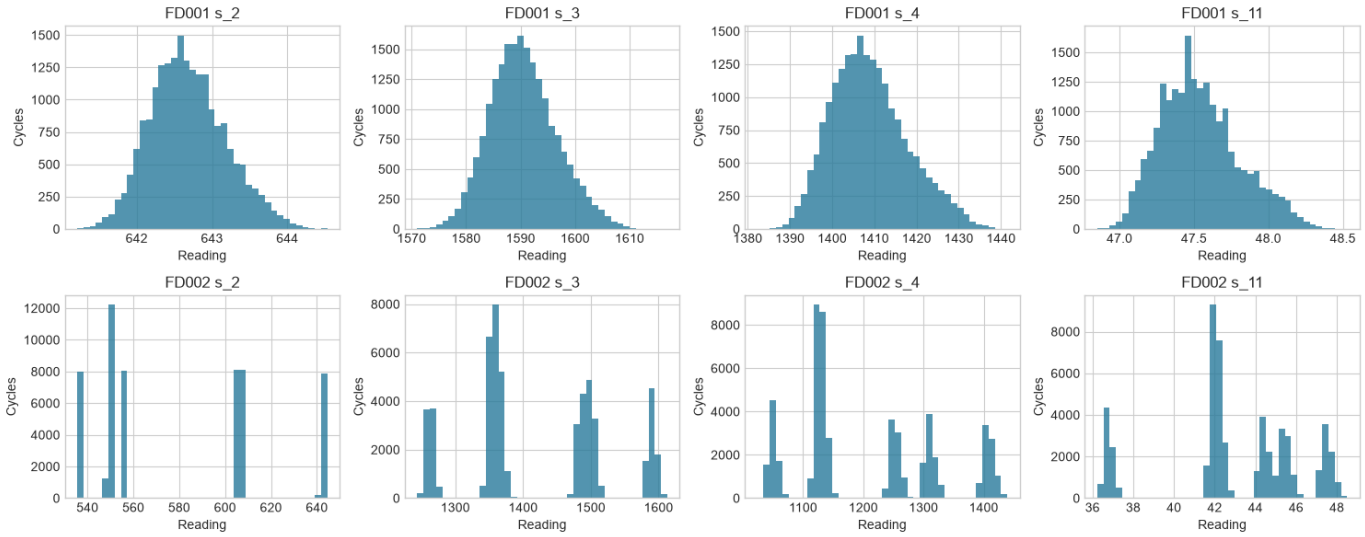


Fig. 2. Raw readings of four sensors over all training cycles in FD001 (top, one operating condition) and FD002 (bottom, six operating conditions).



Fig. 3. Mean FD001 training reading of four sensors in 20 bins of the fraction of each engine's observed life.

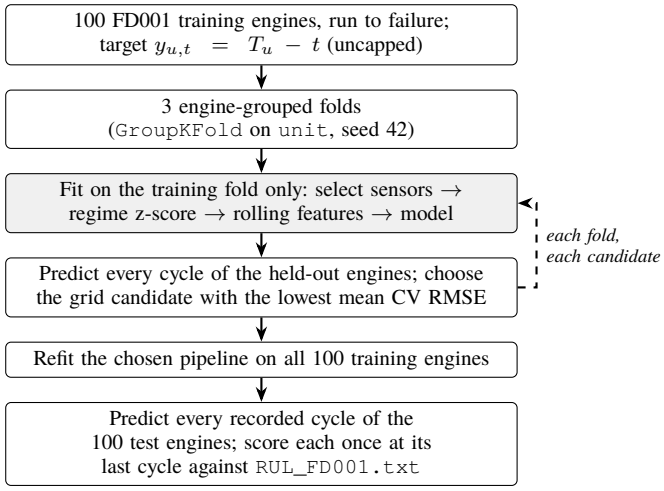


Fig. 4. Evaluation protocol. Every data-dependent preprocessing step is part of the pipeline (shaded), so it is refitted with the model inside each training fold.

on both sides of a split and overstate accuracy. All validation therefore splits by engine: the 100 training engines are divided into three folds with `GroupKFold` on the engine identifier and a fixed seed of 42, and every engine is held out exactly once. Cross-validation scores every cycle of the held-out engines, whose RUL ranges up to 361 cycles, whereas the test set is scored at last observed cycles whose RUL is between 7 and 145. Cross-validation error is therefore used only to choose hyperparameters, not as an estimate of test error.

C. Leakage Control

Every step that learns from data is fitted on training rows only. Sensor selection, the operating-regime clusters and their per-regime statistics are part of a scikit-learn pipeline whose final step is the model, so a grid search refits them inside each training fold, and the final model refits them on all training engines before transforming the test engines. Rolling features use only the current and earlier cycles of the same engine, so they are computed the same way for a complete training trajectory and for a truncated test engine. Unit tests check these properties: folds never share an engine, rolling features ignore later cycles, test rows are scaled with training statistics, and the model never receives the true RUL.

D. Feature Sets and Tuning

Each model is trained on two inputs. The raw set contains the cycle, the three operating settings and all 21 sensors (25 predictors). The engineered set is the 155 predictors of

Section V. The engineered set is the Phase 1 method, fixed before the test set was scored, and the raw set is the comparison that measures what feature engineering adds. Each model is tuned by grid search over the candidates in Section VI, selected by mean CV RMSE, and the best candidate is refitted on all 100 training engines.

E. Official Test Evaluation

The fitted pipeline predicts every recorded cycle of each test engine from that engine's complete history, so that the rolling features at the last cycle have their past. Each engine is then scored once, at its last cycle, against the true RUL in `RUL_FD001.txt`. All 100 test engines are scored for every model, and the per-engine predictions are saved with the metrics so that later phases can compare models engine by engine.

F. Metrics

With $d_u = \hat{y}_u - y_u$ the error at the last cycle of test engine u, we report the RMSE, the MAE and the asymmetric score of the PHM08 challenge [3],

$$S = \sum_u \begin{cases} e^{-d_u/13} - 1, & d_u < 0, \\ e^{d_u/10} - 1, & d_u \geq 0, \end{cases} \tag{1}$$

which penalizes late predictions ($d_u > 0$, more life predicted than remains) more than early ones, because an unexpected failure costs more than early maintenance. S is a sum, not a mean, and grows exponentially, so one large overestimate can dominate it; we always read it together with RMSE and MAE. Following the health-stage view of prognostics [2], errors are also reported by degradation stage of the true RUL: late ($y \leq 50$), mid ($50 < y \leq 100$) and early ($y > 100$). The FD001 test set has 33, 34 and 33 engines in these stages.

G. Implementation and Reproducibility

All shared code is in a Python package (`rul`) with 99 unit tests. Notebooks 01 to 04 run from the raw NASA files to the saved metrics, figures and per-engine predictions. Every stochastic step uses seed 42, and repeated runs reproduce the saved results exactly.

V. Feature Engineering

Phase 1 represents each engine cycle by a fixed-length vector of operating context, sensor levels and causal summaries of recent history. This follows the separation of health-indicator construction from RUL prediction discussed by Lei et al. [2]. Wang et al. [9] estimate remaining life by matching degradation patterns to historical engines; our regressors instead receive explicit summaries of those patterns. The definitions below describe the implementation in `rul.features` and the completed feature-engineering notebook.

A. Sensor Selection and Normalized Readings

Sensor selection uses raw training-fold rows before normalization. The default rule removes only exactly constant channels; it does not assume that every low-variance sensor is uninformative. An optional relative-standard-deviation cutoff compares the population standard deviation with $\max(|\text{mean}|, 1)$ and must be chosen using training data only. On all official FD001 training engines, the default rule removes sensors $s_1, s_5, s_{10}, s_{16}, s_{18}, s_{19}$, retaining 15 channels, including s_6, which is nearly but not exactly constant (two distinct values). The retained list is reused for held-out engines. This reduces redundant inputs without supervised feature selection. Khelif et al. [11] motivate selecting informative sensors for direct RUL regression, although their wrapper selection differs from our variability filter.

Operating conditions can shift sensor levels independently of degradation. K-means is fitted to the three unscaled operating settings from training rows, with one cluster for FD001/FD003 and six for FD002/FD004. The implementation uses seed 42 and ten initializations. Each reading is assigned to its nearest learned centroid, giving regime r, and transformed as

$$z_{u,t,s} = \frac{x_{u,t,s} - \mu_{r,s}}{\sigma_{r,s}}, \tag{2}$$

where u, t, s denote engine, cycle and sensor, and the mean and population standard deviation are fitted within training regime r. A zero standard deviation uses a scale of one. Retained instantaneous $z_{u,t,s}$ values remain predictors, preserving current condition information that averaging may hide. The FD002 training-only comparison for s_2, s_3, s_{11} reduces regime-mean dispersion from about one global standard deviation to numerical zero. This checks removal of training mean offsets; it does not establish a prediction gain.

B. Cycle and Operating Context

The current `cycle` supplies an observed age proxy, allowing models to relate accumulated operation to remaining life. It contains no final-lifetime information, so engines of the same age can still receive different predictions from their sensors. The original `op_1`, `op_2`, `op_3` settings and learned `regime` are also retained to describe the current operating context after sensor normalization. Regime is stored as an integer in the matrix; cluster numbers are arbitrary identifiers with no physical ordering. FD001 has a constant regime column, retained for a consistent schema.

C. Rolling Degradation Features

For each retained normalized sensor, trailing windows of $w \in \{5, 10, 20\}$ observations summarize short and longer histories. Let $\mathcal{H}_{u,t,w}$ contain the current and up to $w - 1$ preceding observations of engine u, and let $n = |\mathcal{H}_{u,t,w}|$. Rows are sorted by engine and cycle for calculation, then restored to their input order. Windows never cross engines, use no future readings, and use the available history at startup.

1) Rolling mean: The local level is

$$m_{u,t,s,w} = \frac{1}{n} \sum_{i \in \mathcal{H}_{u,t,w}} z_{u,i,s}. \tag{3}$$

Averaging smooths cycle-to-cycle fluctuations. Short windows respond faster; long windows smooth more strongly but can delay changes. These window sizes are implementation defaults, not claimed optimal values.

2) Rolling slope: The least-squares trend over the same history is

$$b_{u,t,s,w} = \frac{\sum_{i \in \mathcal{H}_{u,t,w}} (c_i - \bar{c})(z_{u,i,s} - m_{u,t,s,w})}{\sum_{i \in \mathcal{H}_{u,t,w}} (c_i - \bar{c})^2}, \tag{4}$$

where c_i is the recorded cycle and $\bar{c}$ its window mean. The slope is zero for a single observation. It captures direction and rate of change in normalized sensor units per cycle, complementing the local level; short-window slopes can be sensitive to noise.

3) Signed health change: The engine-relative change is

$$h_{u,t,s,w} = m_{u,t,s,w} - z_{u,t_0,s}, \tag{5}$$

where t_0 is that engine's first observed cycle. This provides a baseline relative to each engine's initial observed condition. Unlike a calibrated health index, it need not be monotonic or lie in $[0, 1]$; the first observation is not assumed fault-free. It is zero initially, and its sign depends on the sensor's degradation direction. Rolling summaries use regime-normalized readings and may span regime changes within an engine; they do not restart at each change of regime.

D. Feature Matrix and Leakage Control

With S retained sensors and W window sizes, the predictor count is $5 + S(1 + 3W)$: cycle, three settings, regime, S instantaneous readings and $3SW$ rolling features. The FD001 export therefore has 155 predictors for $S = 15$, $W = 3$, with 20,631 training rows and 13,096 test rows across 100 engines in each split. The notebook writes aligned train/test CSVs and a JSON manifest under `data/processed/`; the manifest records sensors, predictor order, windows, regime count, seed and fitting scope. Engine identifier `unit` is metadata, and uncapped training RUL is a target; neither is a predictor. Test features contain no target.

Final exports fit selection and normalization on all official training engines. For engine-grouped cross-validation, both must instead be refitted within each training fold; globally exported features cannot be reused for tuning. Rolling features are computed independently within each split using only observed engine history. Official test scoring uses each engine's last observed cycle and the corresponding NASA RUL label. The notebook's training correlations support exploratory interpretation only: the strongest is `cycle` itself ($\rho = -0.79$), several rolling means have absolute Spearman correlations near 0.75 with RUL, and slopes are noisier. These observations do not establish held-out accuracy or a benefit from any individual feature group; that requires model evaluation and ablations.

VI. Models and Theory

Phase 1 treats RUL estimation as tabular regression. Each engine cycle is a predictor vector $\mathbf{x}_{u,t}$, either the raw cycle, settings and 21 sensors or the engineered matrix of Section V, and the target is the uncapped training RUL $y_{u,t}$. Three model families are compared under one protocol: a Random Forest [15], XGBoost [16] and an Elastic Net [18]. Each is wrapped in a scikit-learn pipeline whose feature step (sensor selection, regime normalization, rolling features and, for the Elastic Net, scaling) is refitted inside every training fold, so tuning never sees statistics from the engines it is scored on. All three are fitted by squared error and tuned by grid search over three engine-grouped folds with seed 42, using cross-validated RMSE as the selection criterion. Under squared error, each model estimates the conditional mean $\mathbb{E}[y \mid \mathbf{x}]$. The asymmetric NASA score is reported but is not optimized, so the models are not deliberately biased towards early predictions.

A. Random Forest

A regression tree partitions the predictor space with axis-aligned splits. Each split is chosen greedily to minimize the squared error of the two children, and a leaf predicts the mean target of the training rows it holds. A deep tree has low bias but high variance: small changes in the training engines can change the early splits and, with them, the whole tree. A Random Forest [15] grows B such trees, each on a bootstrap sample of rows and with only a random subset of predictors considered at every split, and averages their predictions. If each tree has variance σ^2 and two trees have pairwise correlation ρ, the variance of the average is [22]

$$\mathrm{Var}\left[\frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})\right] = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \tag{6}$$

More trees remove the second term, while random predictor subsets lower ρ and therefore the first term, which more trees cannot reduce.

The method suits C-MAPSS for three reasons. Splits capture non-linear and interacting sensor effects without a specified functional form, such as a sensor that only matters late in life. Trees are invariant to monotone rescaling, so they need no standardization. Because each prediction is an average of training targets, it stays within the observed RUL range and is never negative. The same property means a forest cannot extrapolate beyond the longest training life. The implementation keeps scikit-learn's unlimited depth and bootstrap sampling, sets `max_features`=0.5 so that each split considers half the predictors, and tunes the number of trees $B \in \{50, 100\}$ and the minimum leaf size $\in \{1, 3\}$.

B. XGBoost

Gradient boosting [17] builds an additive model $\hat{y}^{(M)}(\mathbf{x}) = \sum_{m=1}^{M} \eta f_m(\mathbf{x})$, where each new tree f_m is fitted to the negative gradient of the loss of the current ensemble. With squared error, this gradient is the residual, so every tree corrects what the previous trees still get wrong. Where a

forest reduces variance by averaging independent deep trees, boosting reduces bias by adding shallow, dependent trees. XGBoost [16] fits each tree using a second-order expansion of the loss and adds an explicit complexity penalty

$$\Omega(f) = \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2, \tag{7}$$

where T is the number of leaves and w_j the leaf values. For squared error the optimal leaf value is $w_j^* = \sum_{i \in I_j} r_i / (|I_j| + \lambda)$, the mean residual r_i of the $|I_j|$ rows in leaf j shrunk towards zero, so small leaves move the prediction less.

XGBoost suits the problem for the same non-linearity reasons as the forest and usually reaches lower bias, at the cost of more hyperparameters that interact. The implementation uses the squared-error objective and the histogram tree method. It keeps the library defaults for the learning rate ($\eta = 0.3$), $\lambda = 1$, $\gamma = 0$ and row and column subsampling (none), and tunes the number of trees $M \in \{100, 200\}$ and the maximum depth $\in \{3, 5\}$.

C. Elastic Net

The Elastic Net [18] is linear regression with a combined penalty. In the parameterization used by scikit-learn it solves

$$\min_{\beta_0, \boldsymbol{\beta}} \frac{1}{2N} \sum_{i=1}^{N} \big(y_i - \beta_0 - \mathbf{x}_i^\top \boldsymbol{\beta}\big)^2 + \alpha\rho\|\boldsymbol{\beta}\|_1 + \frac{\alpha(1-\rho)}{2}\|\boldsymbol{\beta}\|_2^2, \tag{8}$$

where α sets the overall strength and the ratio ρ moves between ridge ($\rho = 0$) and lasso ($\rho = 1$). The ℓ_1 term sets some coefficients exactly to zero, giving a sparse model. The ℓ_2 term stabilizes the solution when predictors are correlated and produces the grouping effect: correlated predictors receive similar coefficients instead of one being selected arbitrarily. C-MAPSS sensors that respond to the same degradation are strongly correlated, and the engineered matrix adds three rolling summaries of each sensor at three windows, so this property is directly relevant. Because the penalty depends on coefficient size, predictors are standardized inside each training fold before fitting.

The Elastic Net is the linear reference. If it matched the trees, non-linear models would be unnecessary; its coefficients are also directly interpretable. Its limitations follow from linearity. It cannot represent a flat RUL relationship early in life followed by a steep decline, and its predictions are unbounded, so a test engine can receive a negative RUL. The implementation tunes $\alpha \in \{0.1, 1.0\}$ and $\rho \in \{0.2, 0.8\}$ and allows 20,000 coordinate-descent iterations.

D. Bias, Variance and Regularization

For squared error, the expected error of a fitted model $\hat{f}$ at a point $\mathbf{x}$ splits into three parts [22], [23]:

$$\mathbb{E}\big[(y - \hat{f}(\mathbf{x}))^2\big] = \sigma^2(\mathbf{x}) + \big(\mathbb{E}[\hat{f}(\mathbf{x})] - f(\mathbf{x})\big)^2 + \mathrm{Var}\big[\hat{f}(\mathbf{x})\big], \tag{9}$$

the irreducible noise, the squared bias and the variance, where $f(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$ and the expectation is over training sets. Each tuned hyperparameter moves a model along this trade-off. In the forest, a larger minimum leaf size averages more rows per leaf, raising bias and lowering variance, and `max_features` lowers the correlation ρ. In XGBoost, greater depth and more trees lower bias and raise variance, while the learning rate and λ shrink each tree's contribution. In the Elastic Net, a larger α shrinks all coefficients, accepting bias for lower variance.

TABLE III
GRID-SEARCH SELECTIONS ON THE FD001 TRAINING ENGINES (THREE ENGINE-GROUPED FOLDS, SEED 42). CV RMSE IS THE MEAN ± STANDARD DEVIATION OVER FOLDS, IN CYCLES.

Model	Features	Selected	CV RMSE
Random Forest	raw	$B = 100$, leaf 3	37.6 ± 3.8
Random Forest	engineered	$B = 100$, leaf 1	36.4 ± 5.0
XGBoost	raw	$M = 100$, depth 3	38.1 ± 3.7
XGBoost	engineered	$M = 200$, depth 5	38.0 ± 5.1
Elastic Net	raw	$\alpha = 0.1$, $\rho = 0.8$	40.7 ± 4.8
Elastic Net	engineered	$\alpha = 1.0$, $\rho = 0.2$	41.8 ± 3.8

Table III lists the configurations selected on all 100 FD001 training engines. On engineered features, cross-validation chose the more flexible end of each tree grid (leaf size 1; depth 5 with 200 trees), while on raw features it chose the simpler end. The Elastic Net chose stronger shrinkage with a larger ridge share on the engineered matrix, consistent with its many correlated rolling features, although the fold variation is too large to treat this as more than suggestive. The differences in CV RMSE between models and between feature sets, at most 5.4 cycles, are about the size of the standard deviation across folds (3.7 to 5.1 cycles), so cross-validation alone does not establish a preferred model.

The uncapped target adds a large noise term. FD001 training engines lived between 128 and 362 cycles. Early in life, before degradation is visible in the sensors, engines of the same age look alike but have very different remaining lives, so $\sigma^2(\mathbf{x})$ is large and no model can remove it. Tree SHAP on the engineered models attributes far more of each prediction to `cycle` than to any sensor feature, which shows that the models fall back on an age-based estimate where sensors carry little signal. Correspondingly, the tree models' test RMSE is two to eight times larger for engines in the early stage than in the late stage. This is why much of the literature caps the training target at a constant early in life [5], [10]: capping removes much of this noise. It also changes the quantity being predicted, so this project keeps the uncapped target as primary and treats capping as an ablation.

Two limitations of the tuning follow from this framing. Each grid has two values per hyperparameter, and several selections lie on a grid boundary, so the optimum may lie outside the searched range. The XGBoost learning rate, which controls variance together with the number of trees, is fixed at its default rather than tuned. Wider grids would let the trade-off be located rather than bracketed.

E. Machine Learning for RUL

Tree ensembles and regularized regressors are established data-driven prognostic tools. Wu et al. [13] compared Random Forests with feed-forward neural networks and support vector

TABLE IV

OFFICIAL FD001 TEST RESULTS (100 ENGINES, LAST CYCLE, UNCAPPED RUL). STAGE COLUMNS GIVE THE RMSE ON THE 33 LATE, 34 MID AND 33 EARLY ENGINES. LOWER IS BETTER; BOLD MARKS THE BEST OF THE SIX MODELS IN EACH COLUMN.

Model	RMSE	MAE	NASA	Late	Mid	Early
Engineered features (Phase 1 method)						
Random Forest	27.55	**17.67**	145,410	7.52	**25.76**	39.49
XGBoost	30.50	19.82	61,202	**5.62**	29.62	43.39
Elastic Net	28.39	23.57	7,777	24.78	32.60	**27.09**
Raw inputs (comparison)						
Random Forest	**26.15**	19.22	8,414	13.50	27.94	32.95
XGBoost	26.84	19.19	7,002	14.32	28.50	33.77
Elastic Net	31.08	25.80	**6,903**	29.71	32.08	31.38

regression for tool-wear prediction on 315 milling tests and found the forest the most accurate. On C-MAPSS, Wang et al. [14] used Random Forest importance to select engine parameters before training a multilayer perceptron, using the forest for feature selection rather than as the final predictor. Mathew et al. [12] compared a range of classical regressors on the same turbofan data, and Khelif et al. [11] used support vector regression for direct RUL estimation. Our baselines differ from these studies by evaluating all three families under one engine-grouped protocol with fold-fitted preprocessing, which provides the reference for the Phase 2 sequence models.

VII. RESULTS

All results are on the official FD001 test set: 100 engines, each scored once at its last recorded cycle against the uncapped true RUL, unless stated otherwise. The cross-validated configurations behind them are in Table III.

A. Overall Test Error

Table IV lists the six models. With the engineered features, the Phase 1 method, Random Forest reaches an RMSE of 27.55 cycles and an MAE of 17.67 cycles, the lowest MAE of all six models. No model is best on every metric: Random Forest on raw inputs has the lowest RMSE (26.15), and the raw Elastic Net the lowest NASA score. The engineered features lower RMSE only for the Elastic Net (31.08 to 28.39) and raise it for both tree models, by 1.4 cycles for Random Forest and 3.7 for XGBoost. All six RMSEs lie within five cycles of each other on a single set of 100 engines without confidence intervals, so we do not read them as a ranking. Test RMSE (26–31) is well below CV RMSE (36–42) because cross-validation scores every training cycle, including RUL values far above the test range (Section IV).

B. Error by Degradation Stage

The overall numbers hide two opposite effects (Table IV, Fig. 5). Near failure, on the 33 engines with $y \leq 50$, the engineered features roughly halve the tree models' error: Random Forest falls from 13.50 to 7.52 and XGBoost from 14.32 to 5.62. This is the stage in which maintenance is scheduled, and where the rolling means and slopes track the accelerating degradation described in Section III. On the 33 engines with $y > 100$, the engineered tree models are worse

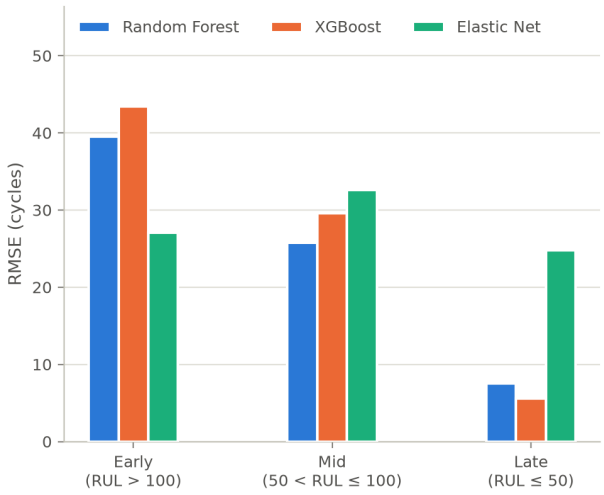


Fig. 5. Test RMSE by degradation stage of the true RUL for the engineered models.

than their raw counterparts (Random Forest 39.49 against 32.95, XGBoost 43.39 against 33.77). The Elastic Net shows the reverse pattern: its late-stage RMSE of 24.78 is the worst of the engineered models, because a linear function cannot follow the sharp fall of RUL at the end of life, but it is the most accurate early.

The early-stage errors are mostly overestimates. An engine whose sensors still look healthy receives a long predicted life, typical of training engines that lived 128 to 362 cycles, yet no test engine had more than 145 cycles left at its last observed cycle. The engineered Random Forest overestimates 9 engines by more than 50 cycles. Tree SHAP attributes far more of each engineered prediction to `cycle` than to any sensor feature (Section VI), so for young engines the models fall back on an age-based estimate: this is the irreducible early-life noise of the uncapped target.

C. Per-Engine Predictions

Fig. 6 shows the same pattern engine by engine: the predictions are tight below about 50 cycles of true RUL and spread widely above about 80. The Elastic Net predicts negative RUL for 7 test engines, since its output is not bounded; the predictions are reported without clipping. Fig. 7 follows the engineered Random Forest over the whole recorded history of the four test engines observed longest, chosen by history length rather than by error. These engines started with 270 to 330 cycles of life, more than most training engines had, and early in each record the prediction stays far below the true RUL. The gap narrows as degradation becomes visible: at the last cycle, engines 49 and 62 are predicted within one cycle, while engines 91 and 93 remain underestimated by 12 and 35 cycles, an early and therefore safe error.

D. Sensitivity of the NASA Score

Because the NASA score is an exponential sum over engines, single engines dominate it. Test engine 1 was observed

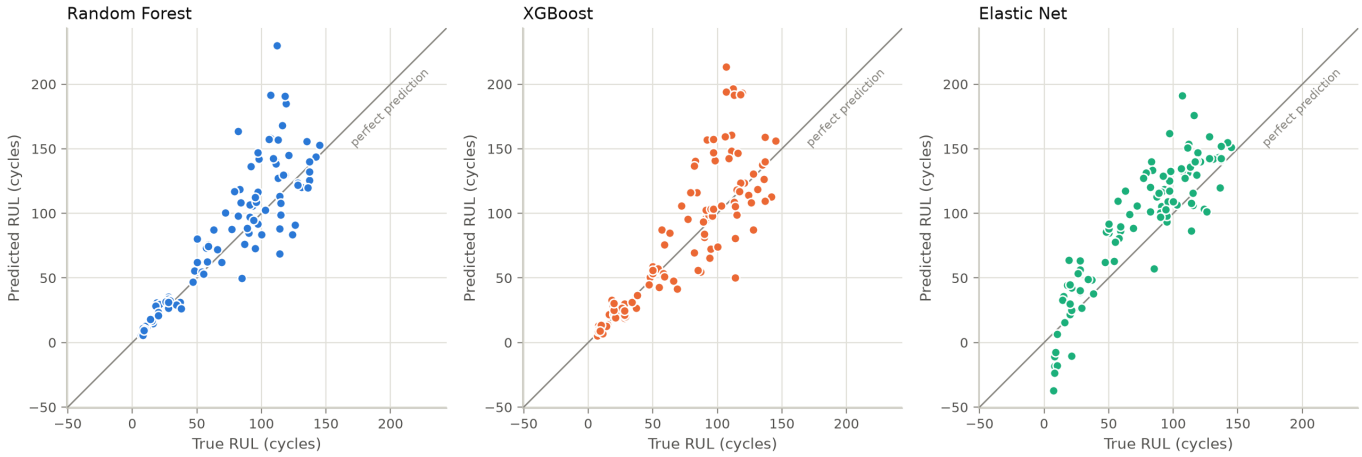


Fig. 6. Predicted against true RUL at the last cycle of each test engine for the engineered models. Points above the diagonal predict more life than the engine had. The panels share one scale so that the Elastic Net's negative predictions remain visible.

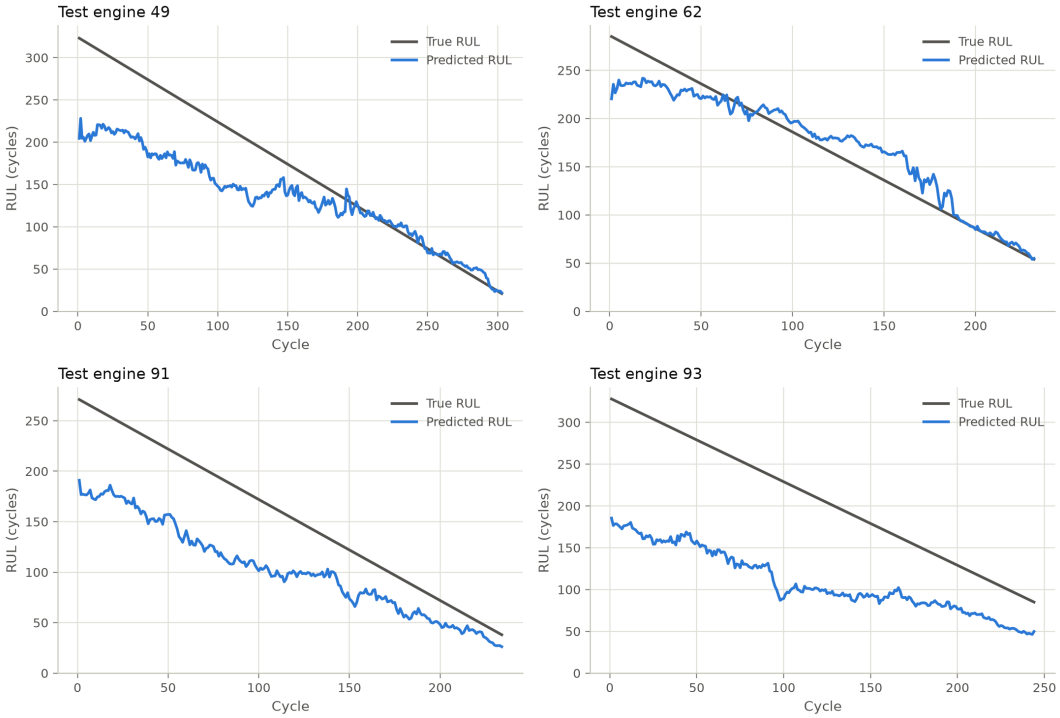


Fig. 7. Engineered Random Forest predictions over the recorded cycles of the four test engines with the longest histories (49, 62, 91 and 93), against the true RUL.

for only 31 cycles and had 112 left; the engineered Random Forest predicts 230. That one overestimate of 118 cycles contributes 133,652 of the model's 145,410, or 92%, while the other 99 engines add 11,758. XGBoost's largest single penalty is 68% of its score. The NASA ranking in Table IV is therefore driven by a handful of young engines, which is why we read it together with RMSE, MAE and the stage errors.

E. Capped-Target Check

To test whether the uncapped target explains the early-stage errors, Random Forest was retrained with the training RUL capped at 125 cycles, using the same grid, folds and seed, and scored against the true, uncapped test RUL (Table V). Capping lowers the RMSE of both feature sets and reverses their order: engineered features now reach 16.21 against 18.66 for raw inputs, with an early-stage RMSE of 20.05 against 23.12 and a late-stage RMSE of 7.39 against 13.44. The comparison between feature sets therefore depends on how RUL is defined. This is a single-model check with a cap taken from the literature, not chosen on the test set; the full ablation over all models, with the cap justified from training

TABLE V
RANDOM FOREST TEST RMSE WITH THE UNCAPPED AND THE CAPPED (125 CYCLES) TRAINING TARGET, SCORED AGAINST THE TRUE, UNCAPPED TEST RUL.

Training target	Engineered	Raw
Uncapped (primary)	27.55	26.15
Capped at 125 (ablation)	16.21	18.66

data, belongs to Phase 2.

VIII. CONCLUSION

Phase 1 established a leakage-controlled protocol and feature-based baselines for RUL prediction on C-MAPSS FD001. With 155 causal engineered features, the Random Forest predicts the RUL of the 100 official test engines with an RMSE of 27.55 cycles and an MAE of 17.67, the lowest MAE of the six models; all six lie within five RMSE cycles of each other. The overall numbers hide the main finding. Near failure, where maintenance is decided, the engineered features roughly halve the error of both tree models, to 7.52 and 5.62 cycles. Early in life, under the uncapped target, they overestimate engines that still look healthy, and a single such engine accounts for 92% of the Random Forest's NASA score. Capping the training target at 125 cycles removes much of this error and reverses the raw-versus-engineered comparison, so conclusions about feature engineering depend on how RUL is defined.

These results have limits. Only FD001, with one operating condition and one fault mode, was evaluated, so regime normalization has not yet been tested where it matters. Each model was tuned over a small grid with three folds, the XGBoost learning rate was not tuned, and cross-validation scores every training cycle rather than last cycles like the test set. Differences between models come from one test set of 100 engines without confidence intervals, and the capped-target result covers only the Random Forest.

Phase 2 keeps the splits, metrics and last-cycle scoring fixed and replaces the fixed rolling summaries with sequence models, an LSTM, a temporal CNN and a Transformer encoder, trained on sensor windows built inside engine boundaries. A hybrid model will fuse a sequence encoder with the engineered-feature branch, targeting the gap identified in Section I. Ablations will cover the capped target with the cap chosen on training data, window length, feature groups and regime normalization on FD002 and FD004, and validation will add repeated seeds, bootstrap confidence intervals and paired per-engine tests using the saved predictions. The bar to beat is the Phase 1 method's test RMSE of 27.55 cycles (26.15 with raw inputs) and its late-stage RMSE of 6 to 8 cycles.

AUTHOR CONTRIBUTIONS

All code, notebooks and report sections are in the project repository, where each file's history shows its author.

- **Gokul VKS** (lead): data loading and RUL labels, evaluation metrics and plots, official test scoring and the results notebook; literature review, introduction, methodology, results and conclusion; report assembly and code review.
- **Milind Bansal**: engine-grouped splits, the exploratory data analysis notebook, operating-regime normalization, and the raw-versus-engineered comparison pipeline with its SHAP analysis.
- **Isha Tomar**: sensor selection, rolling degradation features, the exported feature matrices and the feature-engineering notebook, and the baseline model wrappers.
- **Yashi Agrawal**: the dataset and EDA and the models and theory sections, the consolidated references, the Phase 1 presentation and the capped-target check.
- **Mohammad Affan Anas**: the data download script and data documentation, the setup and reproduction guide, and the feature engineering section.

REFERENCES

[1] A. K. S. Jardine, D. Lin, and D. Banjevic, "A review on machinery diagnostics and prognostics implementing condition-based maintenance," *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483–1510, Oct. 2006.

[2] Y. Lei, N. Li, L. Guo, N. Li, T. Yan, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," *Mechanical Systems and Signal Processing*, vol. 104, pp. 799–834, May 2018.

[3] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *2008 International Conference on Prognostics and Health Management*. Denver, CO, USA: IEEE, Oct. 2008, pp. 1–9.

[4] A. Saxena and K. Goebel, "Turbofan engine degradation simulation data set," NASA Ames Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, USA, 2008. [Online]. Available: https://data.nasa.gov/dataset/c-mapss-aircraft-engine-simulated-data

[5] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)*. IEEE, Jun. 2017, pp. 88–95.

[6] E. Ramasso and A. Saxena, "Performance benchmarking and analysis of prognostic methods for CMAPSS datasets," *International Journal of Prognostics and Health Management*, vol. 5, no. 2, 2014.

[7] X.-S. Si, W. Wang, C.-H. Hu, and D.-H. Zhou, "Remaining useful life estimation – a review on the statistical data driven approaches," *European Journal of Operational Research*, vol. 213, no. 1, pp. 1–14, Aug. 2011.

[8] A. Saxena, J. Celaya, E. Balaban, K. Goebel, B. Saha, S. Saha, and M. Schwabacher, "Metrics for evaluating performance of prognostic techniques," in *2008 International Conference on Prognostics and Health Management*. IEEE, Oct. 2008, pp. 1–17.

[9] T. Wang, J. Yu, D. Siegel, and J. Lee, "A similarity-based prognostics approach for remaining useful life estimation of engineered systems," in *2008 International Conference on Prognostics and Health Management*. IEEE, Oct. 2008, pp. 1–6.

[10] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in *2008 International Conference on Prognostics and Health Management*. IEEE, Oct. 2008, pp. 1–6.

[11] R. Khelif, B. Chebel-Morello, S. Malinowski, E. Laajili, F. Fnaiech, and N. Zerhouni, "Direct remaining useful life estimation based on support vector regression," *IEEE Transactions on Industrial Electronics*, vol. 64, no. 3, pp. 2276–2285, Mar. 2017.

[12] V. Mathew, T. Toby, V. Singh, B. M. Rao, and M. G. Kumar, "Prediction of remaining useful lifetime (RUL) of turbofan engine using machine learning," in *2017 IEEE International Conference on Circuits and Systems (ICCS)*. IEEE, Dec. 2017, pp. 306–311.

[13] D. Wu, C. Jennings, J. Terpenny, R. X. Gao, and S. Kumara, "A comparative study on machine learning algorithms for smart manufacturing: Tool wear prediction using random forests," *Journal of Manufacturing Science and Engineering*, vol. 139, no. 7, p. 071018, Jul. 2017.

[14] H. Wang, D. Li, D. Li, C. Liu, X. Yang, and G. Zhu, "Remaining useful life prediction of aircraft turbofan engine based on random forest feature selection and multi-layer perceptron," *Applied Sciences*, vol. 13, no. 12, p. 7186, Jun. 2023.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. San Francisco, CA, USA: ACM, Aug. 2016, pp. 785–794.

[17] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001.

[18] H. Zou and T. Hastie, "Regularization and variable selection via the elastic net," *Journal of the Royal Statistical Society Series B: Statistical Methodology*, vol. 67, no. 2, pp. 301–320, Apr. 2005.

[19] G. Sateesh Babu, P. Zhao, and X.-L. Li, "Deep convolutional neural network based regression approach for estimation of remaining useful life," in *Database Systems for Advanced Applications (DASFAA 2016)*, ser. Lecture Notes in Computer Science, vol. 9642. Cham: Springer, 2016, pp. 214–228.

[20] X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," *Reliability Engineering & System Safety*, vol. 172, pp. 1–11, Apr. 2018.

[21] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[22] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed., ser. Springer Series in Statistics. New York, NY, USA: Springer, 2009.

[23] S. Geman, E. Bienenstock, and R. Doursat, "Neural networks and the bias/variance dilemma," *Neural Computation*, vol. 4, no. 1, pp. 1–58, Jan. 1992.